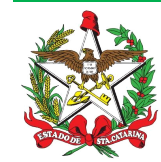




Programação Web

Eventos e Listeners em JavaScript

Professor: Euclides Paim
euclidespaim@gmail.com



Desenvolvimento Web I

Sumário

- **Eventos e Listeners em JavaScript**
 - O DOM (Document Object Model)
 - Selecionando elementos
 - Manipulando conteúdo, atributos, classes e estilo
 - Programação orientada a eventos
 - Formas de registrar manipuladores
 - atributo inline, propriedade *on**, *addEventListener*
 - Tipos de eventos
 - mouse, teclado, formulário, janela e documento
 - O objeto *event*
 - *target*, *currentTarget*, *this*, *preventDefault()*
 - Propagação de eventos
 - *capturing*, *target*, *bubbling*, *stopPropagation()*
 - Delegação de eventos
 - *removeEventListener* e opções do listener
 - Boas práticas
- **Exercícios e Resumo**



Desenvolvimento Web

Introdução

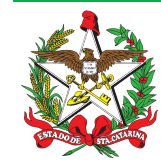
- O que estudamos até agora?
 - **HTML** — elementos, atributos, classes, ids, formulários e elementos semânticos;
 - **CSS** — seletores, box model, posicionamento, transformações e transições;
 - **Fundamentos de JavaScript** — instruções, sintaxe, variáveis, constantes, operadores, tipos de dados, *typeof*, dados primitivos e complexos.
- Até aqui, nosso JavaScript foi um programa **isolado**: ele calcula, compara e armazena valores, mas **não conversa com a página**.
- A partir de agora vamos fazer o JavaScript **ler e alterar a página** e, principalmente, **reagir às ações do usuário** — é isso que chamamos de **eventos** e **listeners**.



Desenvolvimento Web

O DOM

Document Object Model — a ponte entre o JavaScript e a página



Desenvolvimento Web

O que é o DOM

- Quando o navegador carrega uma página HTML, ele não guarda o arquivo como texto: ele constrói na memória uma **estrutura de objetos** que representa o documento.
- Essa estrutura é o **DOM** — *Document Object Model* (Modelo de Objetos do Documento).
- Cada elemento HTML vira um **objeto** (um **nó**, ou *node*), com propriedades e métodos que podemos ler e alterar pelo JavaScript.
- O DOM é uma **API**: uma interface que o navegador oferece para que linguagens de script manipulem o documento.
- Consequência importante:
 - alterar o DOM **altera imediatamente** o que o usuário vê;
 - o arquivo .html **não é modificado** — a mudança acontece apenas na memória do navegador.



Desenvolvimento Web

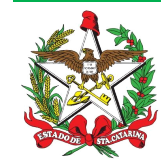
A árvore DOM

- O DOM organiza o documento como uma **árvore hierárquica** de nós:

```
<html>
  <head>
    <title>Minha página</title>
  </head>
  <body>
    <h1 id="titulo">Olá</h1>
    <button class="btn">Clique</button>
  </body>
</html>
```

- Vira a árvore:

```
document
├── html
│   ├── head ─ title ─ "Minha página"
│   └── body ─ h1#titulo ─ "Olá"
│                       └── button.btn ─ "Clique"
```



Desenvolvimento Web

O objeto document

- O objeto **document** é a **porta de entrada** do DOM: ele representa a página inteira e é a partir dele que buscamos os elementos.
- Ele é uma propriedade do objeto global **window** (o navegador), por isso está sempre disponível em qualquer script da página.

```
console.log(document.title);    // título da página  
console.log(document.body);    // elemento <body>  
console.log(document.URL);     // endereço da página
```

- Antes de reagir a um evento, precisamos **selecionar** o elemento envolvido. É o que veremos a seguir.



Desenvolvimento Web

Selecionando elementos — métodos clássicos

- **getElementById()** — busca **um único** elemento pelo atributo *id*:

```
const titulo = document.getElementById("titulo");
```
- **getElementsByClassName()** — retorna uma **coleção** de elementos com a mesma classe:

```
const botoes = document.getElementsByClassName("btn");  
console.log(botoes[0]); // primeiro elemento da coleção
```
- **getElementsByTagName()** — retorna uma coleção pelo nome da tag:

```
const paragrafos = document.getElementsByTagName("p");
```
- Obs.: os métodos no **plural** retornam uma coleção, **nunca** um elemento sozinho — mesmo que exista apenas um resultado.



Desenvolvimento Web

Selecionando elementos — *querySelector*

- Os métodos modernos aceitam **seletores CSS**, os mesmos que já usamos em folhas de estilo:
- **querySelector()** — retorna o **primeiro** elemento que casa com o seletor, ou *null*:

```
const titulo = document.querySelector("#titulo");  
const primeiro = document.querySelector(".btn");  
const link = document.querySelector("nav ul li a");
```

- **querySelectorAll()** — retorna **todos** os elementos que casam, em uma *NodeList*:

```
const botoes = document.querySelectorAll(".btn");  
botoes.forEach(function (b) {  
    console.log(b.textContent);  
});
```

- Na prática, **querySelector** e **querySelectorAll** cobrem quase todos os casos.

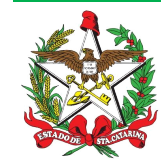


Desenvolvimento Web

Métodos de seleção — resumo

Método	Retorna	Seletor aceito
<code>getElementById()</code>	1 elemento ou null	id (sem #)
<code>getElementsByClassName()</code>	HTMLCollection	classe (sem .)
<code>getElementsByTagName()</code>	HTMLCollection	nome da tag
<code>querySelector()</code>	1 elemento ou null	seletor CSS
<code>querySelectorAll()</code>	NodeList	seletor CSS

- Dica: se o seletor não encontrar nada, **querySelector** devolve *null* — e tentar usar um listener em *null* gera **TypeError**. Sempre confira o seletor.



Desenvolvimento Web

Manipulando o conteúdo

- **textContent** — lê ou escreve apenas o **texto** do elemento:

```
const h1 = document.querySelector("#titulo");  
h1.textContent = "Bom dia!";
```

- **innerHTML** — lê ou escreve o **HTML interno** (o navegador interpreta as tags):

```
h1.innerHTML = "Bom <strong>dia</strong>!";
```

- Obs.: prefira **textContent** quando o conteúdo vier do usuário. Escrever texto de terceiros com **innerHTML** permite injeção de código (**XSS**) na sua página.



Desenvolvimento Web

Manipulando atributos e valores

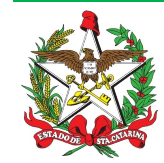
- **getAttribute()** / **setAttribute()** — leem e escrevem qualquer atributo HTML:

```
const img = document.querySelector("#foto");  
console.log(img.getAttribute("src"));  
img.setAttribute("src", "nova.png");  
img.setAttribute("alt", "Nova imagem");
```

- **value** — propriedade usada nos campos de formulário (*input*, *textarea*, *select*):

```
const campo = document.querySelector("#email");  
console.log(campo.value);           // o que o usuário digitou  
campo.value = "";                   // limpa o campo
```

- Guarde bem a propriedade **value**: ela será a base dos eventos de formulário.



Desenvolvimento Web

Manipulando classes e estilo

- **style** — altera o estilo **inline** do elemento (propriedades em *camelCase*):

```
const box = document.querySelector("#box");  
box.style.backgroundColor = "yellow";  
box.style.fontSize = "24px";
```

- **classList** — a forma recomendada: liga e desliga classes já definidas no CSS:

```
box.classList.add("ativo");           // adiciona  
box.classList.remove("ativo");        // remove  
box.classList.toggle("ativo");        // alterna  
box.classList.contains("ativo");      // true ou false
```

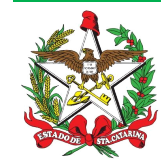
- **Boa prática:** mantenha a aparência no **CSS** e use o JavaScript apenas para **trocar classes**. O código fica mais limpo e o visual centralizado.



Desenvolvimento Web

Eventos

Fazendo a página reagir às ações do usuário



Desenvolvimento Web

O que é um evento

- Um **evento** é um **acontecimento** na página que o navegador percebe e anuncia ao nosso código.
- Exemplos de acontecimentos:
 - o usuário **clica** em um botão;
 - o usuário **digita** uma tecla;
 - o usuário **envia** um formulário;
 - o mouse **entra** ou **sai** de uma área;
 - a página **termina de carregar**;
 - a janela é **redimensionada**.
- O navegador **dispara** o evento; cabe a nós dizer o que fazer quando ele acontecer.



Desenvolvimento Web

Programação orientada a eventos

- Nos programas que escrevemos até agora, as instruções eram executadas **de cima para baixo**, uma única vez, na ordem em que aparecem.
- Na web isso não basta: não sabemos **quando** — nem **se** — o usuário vai clicar em algo.
- Por isso a web usa o paradigma de **programação orientada a eventos**:
 - escrevemos **blocos de código** que ficam à **espera**;
 - quando o evento acontece, o navegador **executa** aquele bloco;
 - enquanto nada acontece, a página fica **ociosa**, sem consumir processamento.
- A função que fica esperando é chamada de **manipulador de evento** (*event handler*) ou **ouvinte** (*event listener*).



Desenvolvimento Web

Handler, listener e callback

- Três termos aparecem o tempo todo na documentação — e às vezes como sinônimos:

Termo	Significado
Event handler	A função que trata o evento (o "o que fazer")
Event listener	O registro que liga um evento a um handler
Callback	Qualquer função passada para ser chamada depois

- Na prática: escrevemos uma **função** e a **registramos** em um elemento para um determinado tipo de evento. O navegador guarda essa ligação e, quando o evento ocorre, **chama a função por nós** — nunca chamamos o handler diretamente.



Desenvolvimento Web

Três formas de registrar um manipulador

- Existem três maneiras de dizer ao navegador "quando isso acontecer, execute aquilo":

Forma	Exemplo	Uso hoje
1. Atributo HTML	<code><button onclick="..."></code>	Evitar
2. Propriedade on*	<code>btn.onclick = fn</code>	Aceitável
3. <code>addEventListener</code>	<code>btn.addEventListener(...)</code>	Recomendado

- Vamos ver as três: as duas primeiras ainda aparecem em código antigo e em exemplos da internet, então precisamos **saber lê-las**. A terceira é a que vamos **usar**.



Desenvolvimento Web

Forma 1 — atributo HTML inline

- O manipulador é escrito **dentro do HTML**, no atributo *on* + nome do evento:

```
<button onclick="alert('Você clicou!')">Clique aqui</button>
```

```
<button onclick="saudar()">Bom dia</button>
```

```
<script>
  function saudar() {
    alert("Bom dia, turma!");
  }
</script>
```

- Funciona, mas **mistura estrutura (HTML) com comportamento (JS)** — exatamente o problema que o CSS externo resolveu para a apresentação.



Desenvolvimento Web

Forma 2 — propriedade `on*` do elemento

- Aqui o HTML fica limpo e a ligação é feita pelo JavaScript:

```
<button id="btn">Clique aqui</button>
```

```
<script>
```

```
  const btn = document.querySelector("#btn");
```

```
  btn.onclick = function () {
```

```
    alert("Você clicou!");
```

```
  };
```

```
</script>
```

- Melhor que a forma 1: separa HTML de JavaScript. Mas tem uma limitação séria...



Desenvolvimento Web

Forma 2 — a limitação

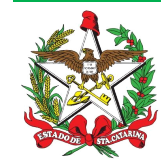
- A propriedade **onclick** é uma **variável**: guarda **um único valor**. Atribuir de novo **substitui** o manipulador anterior.

```
btn.onclick = function () {  
    console.log("Primeiro");  
};
```

```
btn.onclick = function () {  
    console.log("Segundo");    // sobrescreve o primeiro  
};
```

// Ao clicar, imprime apenas: Segundo

- Em páginas reais, com scripts e bibliotecas diferentes tocando no mesmo elemento, isso causa bugs difíceis de encontrar.



Desenvolvimento Web

Forma 3 — `addEventListener()`

- É o método padrão do DOM para registrar manipuladores — e o que vamos usar:

`elemento.addEventLisTener(tipo, funcao, opcoes);`

Parâmetro	Descrição
tipo	Nome do evento, como string: "click", "keydown"...
funcao	Função a executar quando o evento ocorrer
opcoes	Opcional: objeto de configuração ou booleano

- Atenção: o tipo é escrito **sem o prefixo *on*** e entre aspas — "click", e não "onclick".



Desenvolvimento Web

addEventListener() — primeiro exemplo

```
<button id="btn">Clique aqui</button>  
<p id="saida"></p>
```

```
<script>  
  const btn    = document.querySelector("#btn");  
  const saida  = document.querySelector("#saida");  
  
  btn.addEventListener("click", function () {  
    saida.textContent = "Botão clicado!";  
  });  
</script>
```

- Repare: passamos a **função em si**, sem os parênteses. Escrever *minhaFuncao()* **executaria** a função na hora e registraria o seu retorno.



Desenvolvimento Web

addEventListener() — múltiplos ouvintes

- A grande vantagem: um mesmo elemento pode ter **vários** manipuladores para o **mesmo** evento — todos são executados, na ordem de registro.

```
btn.addEventListener("click", function () {  
    console.log("Primeiro");  
});
```

```
btn.addEventListener("click", function () {  
    console.log("Segundo");  
});
```

```
// Ao clicar, imprime: Primeiro  
//                      Segundo
```

- Nada é sobrescrito. Cada trecho do código cuida da sua parte.



Desenvolvimento Web

Função nomeada, anônima ou arrow

- O segundo argumento pode ser escrito de três formas equivalentes:

```
// função nomeada – reaproveitável e removível
```

```
function mostrar() { console.log("Oi!"); }
```

```
btn.addEventListener("click", mostrar);
```

```
// função anônima – a mais comum em exemplos
```

```
btn.addEventListener("click", function () { console.log("Oi!"); });
```

```
// arrow function (ES6) – mais concisa
```

```
btn.addEventListener("click", () => console.log("Oi!"));
```

- Use a **função nomeada** quando precisar **remove** o listener depois ou reaproveitar a lógica em mais de um elemento.



Tipos de eventos

Mouse, teclado, formulário, janela e documento

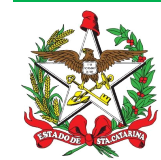


Desenvolvimento Web

Eventos de mouse

Evento	Ocorre quando
click	O usuário clica (pressiona e solta) no elemento
dblclick	O usuário dá um duplo clique
mousedown	O botão do mouse é pressionado
mouseup	O botão do mouse é solto
mousemove	O ponteiro se move sobre o elemento
mouseenter	O ponteiro entra na área do elemento
mouseleave	O ponteiro sai da área do elemento
contextmenu	O usuário aciona o menu de contexto (botão direito)

Fonte: https://developer.mozilla.org/pt-BR/docs/Web/API/Element#eventos_de_mouse



Desenvolvimento Web

Eventos de teclado

Evento	Ocorre quando
keydown	Uma tecla é pressionada (repete se segurada)
keyup	Uma tecla é solta

```
document.addEventListener("keydown", function (event) {  
    console.log("Tecla:", event.key);    // "a", "Enter", "ArrowUp"  
    console.log("Código:", event.code);  // "KeyA", "Enter", "ArrowUp"  
  
    if (event.key === "Escape") {  
        console.log("Fechar a janela!");  
    }  
});
```

- Use **event.key** para saber o **caractere** e **event.code** para a **tecla física**.



Desenvolvimento Web

Eventos de formulário

Evento	Ocorre quando
submit	O formulário é enviado (registrado no ` <code><form></code> `)
input	O valor do campo muda, a cada caractere digitado
change	O valor muda e o campo perde o foco
focus	O campo recebe o foco
blur	O campo perde o foco
reset	O formulário é limpo

- **input** é o evento certo para validação em tempo real; **change** serve melhor para *select* e *checkbox*.



Desenvolvimento Web

Eventos de janela e documento

Evento	Ocorre quando
DOMContentLoaded	O HTML foi lido e o DOM está montado
load	A página e todos os recursos terminaram de carregar
resize	A janela do navegador é redimensionada
scroll	O usuário rola a página ou o elemento
beforeunload	A página está prestes a ser fechada

- Esses eventos costumam ser registrados em **window** ou **document**, e não em um elemento específico.



Desenvolvimento Web

DOMContentLoaded — por que importa

- Se o `<script>` roda **antes** do elemento existir no DOM, `querySelector` devolve **null** e o listener nunca é registrado:

```
<head>
  <script>
    const btn = document.querySelector("#btn");
    btn.addEventListener("click", ...); // TypeError: btn is null
  </script>
</head>
```

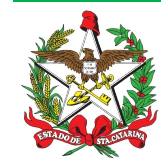
- Duas soluções:
 - colocar o `<script>` **no final do <body>** (mais simples); ou
 - esperar o evento **DOMContentLoaded**:

```
document.addEventListener("DOMContentLoaded", function () {
  // aqui o DOM já está pronto
});
```



O objeto event

Informações sobre o que acabou de acontecer



Desenvolvimento Web

O objeto event

- Quando o navegador chama nosso manipulador, ele passa **automaticamente** um argumento: um objeto com todos os detalhes do que aconteceu.
- Basta declarar o parâmetro para recebê-lo — o nome é livre (*event*, *e* e *evt* são os mais usados):

```
btn.addEventListener("click", function (event) {  
    console.log(event);           // objeto MouseEvent  
    console.log(event.type);      // "click"  
    console.log(event.target);    // <button id="btn">  
});
```

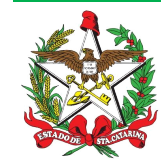
- O tipo do objeto varia conforme o evento: *MouseEvent*, *KeyboardEvent*, *SubmitEvent* — cada um com propriedades próprias.



Desenvolvimento Web

Propriedades e métodos comuns

Membro	Descrição
<code>event.type</code>	Nome do evento: "click", "keydown"...
<code>event.target</code>	Elemento que originou o evento
<code>event.currentTarget</code>	Elemento em que o listener está registrado
<code>event.key</code>	Tecla pressionada (eventos de teclado)
<code>event.clientX / clientY</code>	Coordenadas do ponteiro na janela
<code>event.preventDefault()</code>	Cancela o comportamento padrão do navegador
<code>event.stopPropagation()</code>	Interrompe a propagação do evento



Desenvolvimento Web

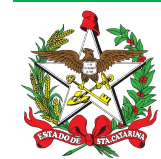
target x currentTarget

- **event.target** — onde o evento **nasceu** (o elemento realmente clicado).
- **event.currentTarget** — onde o listener **está registrado**.

```
<div id="caixa">  
  <button id="btn">Clique</button>  
</div>
```

```
<script>  
  const caixa = document.querySelector("#caixa");  
  
  caixa.addEventListener("click", function (event) {  
    console.log(event.target);           // <button id="btn">  
    console.log(event.currentTarget);    // <div id="caixa">  
  });  
</script>
```

- Essa diferença é a base da **delegação de eventos**, que veremos adiante.



Desenvolvimento Web

A palavra-chave *this* no manipulador

- Em uma **função tradicional**, *this* é o elemento do listener — o mesmo que *event.currentTarget*:

```
btn.addEventListener("click", function () {  
    this.textContent = "Clicado!";    // this = btn  
    this.classList.add("ativo");  
});
```

- Em uma **arrow function**, *this* **não** aponta para o elemento — ela herda o *this* do escopo externo:

```
btn.addEventListener("click", () => {  
    this.textContent = "Clicado!";    // this NÃO é o btn  
});
```

- **Regra prática**: se for usar *this*, escreva uma função tradicional. Com arrow function, use *event.currentTarget*.



Desenvolvimento Web

preventDefault()

- Muitos elementos têm um **comportamento padrão** do navegador:
 - um `<a>` navega para o `href`;
 - um `<form>` é enviado e a página recarrega;
 - um `<input type="checkbox">` marca ou desmarca.
- O método **`event.preventDefault()`** cancela esse comportamento, deixando a decisão com o nosso código:

```
const link = document.querySelector("#externo");

link.addEventListener("click", function (event) {
  event.preventDefault();           // não navega
  console.log("Navegação cancelada");
});
```



Desenvolvimento Web

Exemplo — validação de formulário

```
<form id="cadastro">
  <input type="text" id="nome" placeholder="Seu nome">
  <button type="submit">Enviar</button>
</form>
<p id="aviso"></p>

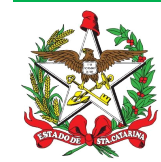
<script>
  const form  = document.querySelector("#cadastro");
  const nome  = document.querySelector("#nome");
  const aviso = document.querySelector("#aviso");

  form.addEventListener("submit", function (event) {
    if (nome.value.trim() === "") {
      event.preventDefault();           // impede o envio
      aviso.textContent = "Informe o seu nome.";
    }
  });
</script>
```



Propagação

Capturing, target e bubbling



Desenvolvimento Web

As três fases da propagação

- Elementos ficam **aninhados** uns dentro dos outros. Ao clicar em um botão dentro de uma *div*, o clique também "acontece" na *div*, no *body*, no *html*...
- O navegador percorre esse caminho em **três fases**:
 - **1. Captura** (*capturing*) — de *window* até o elemento clicado;
 - **2. Alvo** (*target*) — chega ao elemento que originou o evento;
 - **3. Borbulhamento** (*bubbling*) — volta do elemento até *window*.

```
window → document → html → body → div → BUTTON    (captura)
                                   ↓                (alvo)
BUTTON → div → body → html → document → window    (borbulha)
```

- Por padrão, **addEventListener** escuta na fase de **borbulhamento**.



Desenvolvimento Web

Bubbling na prática

```
<div id="externo">
  <div id="interno">
    <button id="btn">Clique</button>
  </div>
</div>

<script>
  document.querySelector("#btn").addEventListener("click", () =>
    console.log("botão"));
  document.querySelector("#interno").addEventListener("click", () =>
    console.log("interno"));
  document.querySelector("#externo").addEventListener("click", () =>
    console.log("externo"));
</script>
```

- Um único clique no botão imprime: **botão** → **interno** → **externo**. É o evento **subindo** pela árvore.



Desenvolvimento Web

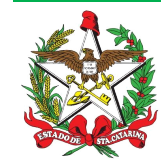
Escutando na fase de captura

- Para escutar na descida, passe o terceiro argumento:

```
// forma booleana (antiga)  
externo.addEventListener("click", handler, true);
```

```
// forma com objeto de opções (recomendada)  
externo.addEventListener("click", handler, { capture: true });
```

- Com *capture: true* nos três *div* do exemplo anterior, a ordem se inverte: **externo** → **interno** → **botão**.
- Na prática, a fase de captura é **pouco usada** — mas entender que ela existe explica muitos comportamentos estranhos em páginas complexas.



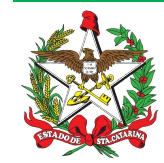
Desenvolvimento Web

stopPropagation()

- O método **event.stopPropagation()** interrompe o caminho do evento: os elementos seguintes **não** recebem o evento.

```
interno.addEventListener("click", function (event) {  
    event.stopPropagation();  
    console.log("interno");    // externo NÃO será avisado  
});
```

- Use com **parcimônia**: um evento interrompido pode quebrar código de outra parte da página que dependia dele.
- Não confunda:
 - **preventDefault()** — cancela a **ação padrão do navegador**;
 - **stopPropagation()** — cancela a **propagação pela árvore**.



Desenvolvimento Web

Delegação de eventos — o problema

- Imagine uma lista com 200 itens, cada um precisando reagir a um clique:

```
const itens = document.querySelectorAll("#lista li");
```

```
itens.forEach(function (item) {  
  item.addEventListener("click", function () {  
    console.log(this.textContent);  
  });  
});
```

- Dois problemas:
 - são **200 listeners** na memória — desperdício de recursos;
 - itens **criados depois** (por exemplo, adicionados pelo usuário) **não terão listener**, porque o *forEach* já rodou.



Desenvolvimento Web

Delegação de eventos — a solução

- Registramos **um único listener no elemento pai** e usamos *event.target* para descobrir quem foi clicado. É a **delegação de eventos**.

```
const lista = document.querySelector("#lista");

lista.addEventListener("click", function (event) {
  if (event.target.tagName === "LI") {
    console.log("Clicou em:", event.target.textContent);
    event.target.classList.toggle("concluido");
  }
});
```

- Vantagens: **um listener** apenas, e itens adicionados depois **já funcionam** — porque o evento deles borbulha até o pai.



Desenvolvimento Web

removeEventListener()

- Para desligar um manipulador, passe **exatamente os mesmos argumentos** usados no registro:

```
function aoClicar() {  
    console.log("Clicou!");  
}
```

```
btn.addEventListener("click", aoClicar);  
btn.removeEventListener("click", aoClicar);    // funciona
```

- Com função **anônima** é **impossível** remover, pois não há como referenciar a mesma função:
- Alternativa: a opção **once**, que remove o listener automaticamente após a primeira execução:

```
btn.addEventListener("click", festejar, { once: true });
```



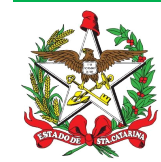
Desenvolvimento Web

Opções do listener

Opção	Valor	Efeito
capture	true / false	Escuta na fase de captura (padrão: false)
once	true / false	Executa uma única vez e se remove
passive	true / false	Promete não chamar preventDefault()
signal	AbortSignal	Permite cancelar o listener depois

`elemento.addEventListener("scroll", aoRolar, { passive: true });`

- *passive: true* é usado em **scroll** e **touchmove**: avisa ao navegador que ele pode rolar a página sem esperar o handler, deixando a rolagem mais fluida.



Desenvolvimento Web

Boas práticas

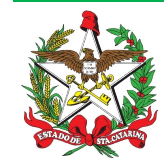
- Prefira sempre **addEventListener** às formas antigas.
- Mantenha o JavaScript em **arquivo externo**, longe do HTML.
- Garanta que o DOM existe antes de registrar listeners (*script* no fim do *body* ou **DOMContentLoaded**).
- Dê **nomes descritivos** aos manipuladores: *aoEnviarFormulario*, *aoClicarNoBotao*.
- Use **delegação** quando houver muitos elementos semelhantes.
- Evite **stopPropagation()** sem necessidade real.
- Prefira alterar **classes** a alterar *style* diretamente.
- Remova listeners que não são mais necessários (*removeEventListener* ou *once*).



Desenvolvimento Web

Exercícios

Hora de colocar a mão no código

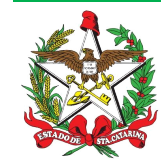


Desenvolvimento Web

Exercício 1 — Contador de cliques

- Crie uma página com um botão e um parágrafo. A cada clique no botão, o parágrafo deve mostrar quantas vezes ele já foi clicado.
- Requisitos:
 - usar **querySelector** e **addEventListener**;
 - guardar a contagem em uma variável *let*;
 - atualizar o texto com **textContent**.
- **Desafio:** adicione um segundo botão "Zerar" que devolve o contador a 0.

```
<button id="contar">Clique</button>  
<button id="zerar">Zerar</button>  
<p id="saida">Cliques: 0</p>
```



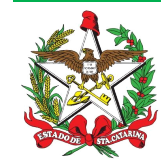
Desenvolvimento Web

Exercício 2 — Lista de tarefas

- Monte uma lista de tarefas com um campo de texto, um botão "Adicionar" e uma `<ul>`.
- Requisitos:
 - ao clicar em **Adicionar**, criar um `<li>` com o texto digitado e limpar o campo;
 - ao clicar em um item da lista, **riscar** a tarefa (alternando uma classe CSS);
 - usar **delegação de eventos** na `<ul>` — os itens são criados depois!

```
// dica para criar o item
const li = document.createElement("li");
li.textContent = campo.value;
lista.appendChild(li);
```

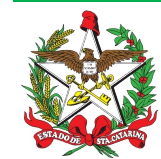
- **Desafio:** não permitir adicionar tarefa com o campo vazio.



Desenvolvimento Web

Exercício 3 — Validação de cadastro

- Crie um formulário com os campos **nome**, **e-mail** e **senha**, e um botão de envio.
- Requisitos:
 - no evento **submit**, validar os três campos;
 - se algum estiver vazio, chamar **preventDefault()** e exibir a mensagem de erro;
 - a senha deve ter no mínimo **8 caracteres**;
 - o e-mail deve conter o caractere **@**;
 - usar o evento **input** para limpar a mensagem de erro assim que o usuário corrigir.
- **Desafio:** destacar o campo inválido com uma classe CSS (borda vermelha).



Desenvolvimento Web

Exercício 4 — Desafio: caça ao botão

- Faça um pequeno jogo:
 - um botão aparece em uma **posição aleatória** dentro de uma área;
 - a cada **clique**, ele muda de lugar e o placar aumenta;
 - um contador regressivo de **20 segundos** encerra a partida;
 - ao fim, exibir a pontuação e um botão "Jogar novamente".

```
// posição aleatória dentro de uma área de 400x300
const x = Math.floor(Math.random() * 400);
const y = Math.floor(Math.random() * 300);
btn.style.left = x + "px";
btn.style.top = y + "px";
```

- **Desafio extra:** encerrar o jogo também quando o usuário pressionar *Esc* (evento **keydown** em *document*).



Desenvolvimento Web I

Resumo

- **Eventos e Listeners em JavaScript**
 - O **DOM** representa a página como uma árvore de objetos manipuláveis
 - **querySelector** e **querySelectorAll** selecionam elementos por seletor CSS
 - **textContent**, **innerHTML**, **setAttribute**, **classList** e **style** alteram o documento
 - A web usa **programação orientada a eventos**: o código espera e reage
 - **addEventListener(tipo, função, opções)** é a forma padrão de registrar manipuladores
 - Permite **vários listeners** no mesmo evento, sem sobrescrever
 - Tipos: mouse, teclado, formulário, janela e documento
 - O objeto **event** traz *type*, *target*, *currentTarget*, *key*, coordenadas...
 - **preventDefault()** cancela a ação padrão; **stopPropagation()** cancela a propagação
 - Fases: **captura**, **alvo** e **borbulhamento** (o padrão é borbulhamento)
 - **Delegação de eventos**: um listener no pai atende muitos filhos
 - **removeEventListener** exige a mesma função; a opção **once** automatiza a remoção
- **Exercícios** — contador, lista de tarefas, validação e caça ao botão



Referências

Referências Básicas

RESIG, John; BIBEAULT, Bear; MARZ, Josip. Secrets of the JavaScript Ninja. 2. ed. Shelter Island: Manning Publications, 2016.

SILVA, Maurício Samy. HTML 5: a linguagem de marcação que revolucionou a web. São Paulo: Novatec, 2011.

GRONER, Loiane. Estrutura de dados e algoritmos com JavaScript. São Paulo: Novatec Editora, 2019.

Referências Complementares

FLANAGAN, David. JavaScript: o guia definitivo. 6. ed. Porto Alegre: Bookman, 2012.

SILVA, Maurício Samy. Criando sites com HTML: sites de alta qualidade com HTML e CSS. São Paulo: Novatec, 2010.

IEPSEN, Edécio Fernandes. Lógica de Programação e Algoritmos com JavaScript. São Paulo: Novatec, 2018.

Referências na Internet

https://developer.mozilla.org/pt-BR/docs/Web/API/Document_Object_Model

<https://developer.mozilla.org/pt-BR/docs/Web/API/EventTarget/addEventListener>

https://www.w3schools.com/js/js_htmlDOM.asp

<https://javascript.info/events>